
title: "Project Report: FitTrack AI" author: "[Student Name]" date: "2026-04-07"

PROJECT REPORT

FitTrack AI – Personal Fitness, Diet, and Attendance Management System

Submitted in partial fulfillment of the requirements for the degree of **[Insert Degree Name, e.g., Bachelor of Technology]**

Submitted by: **[Student Name] [Roll Number/Enrollment Number]**

Submitted to: **[Professor's Name / Department Name] [University / College Name]**

Table of Contents

1. [Abstract](#)
2. [Introduction](#)
3. [System Overview](#)
4. [Detailed Module Explanation](#)
5. [R Programming in the Project](#)
6. [R Code Implementation](#)
7. [Frontend Explanation](#)
8. [Backend Explanation](#)
9. [Database Explanation](#)
10. [Advantages of Using R in This Project](#)
11. [Features and Benefits of the Application](#)
12. [Why This Project is Effective](#)
13. [Future Enhancements](#)
14. [Conclusion](#)
15. [Viva Questions and Answers](#)

1. Abstract

Overview of the App

FitTrack AI is a comprehensive, mobile-first ecosystem designed to consolidate personal health management. It seamlessly integrates fitness tracking, diet planning, and motivational attendance monitoring into a single intuitive platform.

Problem Statement

The modern health and wellness digital landscape is highly fragmented. Users often have to download one application for tracking their workouts, another for logging their daily food intake, and sometimes a third for habit tracking. This fragmentation leads to user fatigue, inconsistent data monitoring, and a higher probability of users abandoning their fitness goals.

Solution Provided

FitTrack AI solves this issue by offering a unified system. It calculates personalized metrics (BMI, BMR, TDEE) to generate tailored diet and workout plans. Furthermore, it incorporates an attendance streak and gamification module to reinforce discipline, ensuring users remain engaged and consistent on their fitness journeys.

2. Introduction

Background of Fitness Apps

Over the last decade, mobile health (mHealth) applications have surged in popularity. However, many applications focus too narrowly on a specific niche, ignoring the holistic nature of fitness which requires a balance of exercise, nutrition, and psychological motivation.

Need for Combining Fitness, Diet, and Attendance

Fitness results are heavily dependent not just on time spent in the gym, but on nutritional intake and strict adherence to a routine. Separating these elements creates a disconnect. By combining them, FitTrack AI provides a complete overview of the user's health profile, correlating dietary intake with workout performance and overall consistency.

Purpose of the Project

The primary purpose of this project is to architect and develop a singular, cohesive software solution that offers robust mathematical processing for health metrics, an engaging user interface, and a lightweight, efficient backend.

3. System Overview

FitTrack AI is built on a modern, robust **Three-Tier Architecture**:

1. Frontend (Client Tier): Flutter

- Built using the Flutter SDK by Google, offering a beautiful, natively compiled, cross-platform application from a single codebase. It provides a rich UI/UX with smooth animations and dynamic state management.

2. Backend (Application Tier): R with Plumber API

- The backend calculation engine leverages the computational prowess of **R**, a language highly specialized in statistical computing. The `plumber` package is utilized to convert R scripts into scalable high-performance REST APIs.

3. Database (Data Tier): SQLite

- A lightweight, offline-first relational database embedded directly within the application, ensuring that the app functions rapidly without requiring constant internet access, with local data persistence.

(Diagram Reference: Mobile Device (Flutter) <--> REST over HTTP (JSON) <--> R API Server (Plumber) <--> Database (SQLite))

4. Detailed Module Explanation

a) User Profile Module

- **Input Data:** Collects critical anthropometric data exactly once during onboarding: Name, Age (years), Gender, Height (cm), Weight (kg), Goal (Lose Weight, Maintain, Gain Muscle), and Activity Level.
- **Storage:** Data is serialized and stored persistently.

- **Flow:** User enters data -> Validated by UI -> Sent to Backend for processing -> Calculated metrics stored in Database.

b) Fitness Module

- **Workout Generation Logic:** Based on the user's goal, the system assigns a specific workout schema. For example, a "Gain Muscle" goal triggers a Push/Pull/Legs (PPL) intermediate routine, while "Lose Weight" triggers a beginner full-body routine with higher reps.
- **Tracking:** Users can tap to complete sets. A built-in rest timer ensures proper recovery between sets.

c) Diet Module

- **Calorie Calculation Logic:** Utilizes the Mifflin-St Jeor equation to calculate Basal Metabolic Rate (BMR), scales it by the activity multiplier to find Total Daily Energy Expenditure (TDEE), and applies a surplus or deficit based on goals.
- **Macro Splitting:** Calculates specific daily grams required for Proteins (based on body weight), Carbs, and Fats.
- **Log System:** Users log meals with specific calorie/macro values, which deducts from their daily remaining targets.

d) Attendance Module

- **Consistency Tracking:** A calendar system where users "Mark Present" or log a "Rest Day".
- **Streak System:** Calculates consecutive days the user has interacted with the app. Missing a day resets the streak, heavily incentivizing daily engagement. Badges are awarded at milestones (3 days, 7 days, 30 days).

5. R Programming in the Project

What is R Programming?

R is a programming language and free software environment designed primarily for statistical computing, data analysis, and graphical visualization.

Why R was used in this project

While node.js or Python are common for backends, R was chosen specifically for its powerful, concise handling of mathematical modeling and arrays. As FitTrack AI heavily relies on anthropometric formulas, data processing, and future predictive analytics (like predicting weight trends), R is the optimal engine for truth.

Topics Used:

- **Variables:** Used to store incoming JSON payload data (`weight` , `height` , etc.).
 - **Conditional Statements (if-else):** Crucial for applying different mathematical formulas based on categorical input, such as Gender impacting the BMR calculation equation.
 - **Functions:** Wrapping the calculation logic into modular, reusable R functions.
 - **Mathematical Calculations:** Executing complex float-point arithmetic accurately for BMI and BMR formulas.
 - **API Creation (`plumber`):** Using special commenting tags (`## @post`) to expose R functions as HTTP endpoints.
-

6. R Code Implementation

Below is the core logic implemented in the backend using R and the `plumber` package.

```
# plumber.R

## @apiTitle FitTrack AI Backend Calculation Engine
## @apiDescription R-based API for deriving physiological fitness metrics

## Calculate User Metrics (BMI, BMR, TDEE, Macros)
## @param weight User weight in kg
## @param height User height in cm
## @param age User age in years
## @param gender User gender (Male/Female)
## @param activity Activity multiplier (e.g., 1.2 to 1.9)
## @param goal User Goal (Lose, Maintain, Gain)
## @post /api/calculate
function(weight, height, age, gender, activity, goal) {

  # 1. Variables formulation & type conversion
  w <- as.numeric(weight)
  h <- as.numeric(height)
  a <- as.numeric(age)
  act <- as.numeric(activity)

  # 2. Mathematical Calculation: Body Mass Index (BMI)
  bmi <- w / ((h / 100) ^ 2)
  bmi <- round(bmi, 2)

  # 3. Conditional Statements: Basal Metabolic Rate (Mifflin-St Jeor)
  if (gender == "Male") {
    bmr <- (10 * w) + (6.25 * h) - (5 * a) + 5
  } else {
    bmr <- (10 * w) + (6.25 * h) - (5 * a) - 161
  }

  # 4. Total Daily Energy Expenditure
  tdee <- bmr * act

  # Modify TDEE based on Goal
  target_calories <- tdee
  if (goal == "Lose weight") {
    target_calories <- tdee - 500 # 500 kcal deficit
  } else if (goal == "Gain muscle") {
    target_calories <- tdee + 300 # 300 kcal surplus
  }
```

```

}

# 5. Functions & Return Structure
# Calculate rough macros
protein_g <- round(w * 2.2) # ~1g per lb of body weight
cals_from_protein <- protein_g * 4
remaining_cals <- target_calories - cals_from_protein
carbs_g <- round((remaining_cals * 0.6) / 4)
fats_g <- round((remaining_cals * 0.4) / 9)

list(
  status = "success",
  data = list(
    bmi = bmi,
    bmr = round(bmr),
    target_calories = round(target_calories),
    macros = list(
      protein = protein_g,
      carbs = carbs_g,
      fats = fats_g
    )
  )
)
}

```

Line-by-Line Explanation:

1. `function(weight, height...)` : Defines the input parameters the API expects.
2. `as.numeric()` : Converts string HTTP inputs securely into mathematical floating-point numbers.
3. `bmi <- ...` : Applies the standard `weight / height^2` algorithm.
4. `if (gender == "Male")` : Applies the gender-specific constant (`+5` for males, `-161` for females) critical to the Mifflin-St Jeor equation.
5. `if (goal == ...)` : Implements the nutritional science logic of creating caloric deficits or surpluses depending on the user's objective.
6. `list(...)` : Compiles the data back into an R list, which `plumber` automatically serializes into JSON for the Flutter frontend to consume.

7. Frontend Explanation

- **Technology: Flutter (Dart).** Flutter was chosen because it renders native arm code for both iOS and Android simultaneously, providing 60/120fps performance.
 - **UI/UX Design Principles:** The application adheres to a "Dark Mode Premium" aesthetic, utilizing neon accents (green/blue gradients) to highlight positive feedback (streaks, ring completion) against a deep dark background (#0b0f19) to reduce eye strain. Glassmorphism and soft shadows give cards depth.
 - **Screens and Navigation:** Implemented a Bottom Navigation Bar routing between Home, Workout, Diet, Attendance, and Profile. Transitions utilize slide-and-fade animations.
 - **User Interaction:** Heavy use of immediate feedback—pressing a button triggers haptic feedback, ripples, and visual cues (confetti on reaching a goal).
-

8. Backend Explanation

- **R + Plumber API:** The backend operates as a microservice. Instead of a heavy monolithic framework like Django, Plumber sets up an HTTP listener.
 - **Data Processing:** It strictly acts as a processing engine. It accepts raw variables, processes them through scientific health formulas, and returns objective targets.
 - **Communication:** Communication occurs asynchronously over HTTP using JSON. Flutter uses the `http` package to `POST` user data to the R server and `await` s the JSON response to update the UI state.
-

9. Database Explanation

- **Type of Database: SQLite** (via `sqflite` plugin in Flutter or local storage equivalents). It is embedded, serverless, and configuration-free.
- **Architecture Setup:** Operates on an offline-first principle.
- **Tables:**
 - `User_Table` : Stores `id`, `name`, `age`, `weight`, `target_calories`, etc.
 - `Attendance_Table` : Stores `date` (PRIMARY KEY), `status` (Present/Rest) .
 - `Meal_Log_Table` : Stores `id`, `date`, `meal_name`, `calories`, `protein` .
 - `Weight_Log_Table` : Stores `date`, `weight_kg` for generating progress graphs.

10. Advantages of Using R in This Project

1. **Precision Mathematics:** R handles complex float operations with extreme precision natively.
 2. **Rapid Prototyping:** Writing mathematical models in R is significantly faster and requires fewer lines of code compared to Java or Dart.
 3. **API Simplicity:** The `plumber` library makes deploying these models as REST APIs a matter of just a few comment annotations.
 4. **Future AI Ready:** R is a premier language for Machine Learning. Future features like "Predicting weight goal dates based on trend lines" can easily utilize R's built-in regression models (`lm()`) without changing the tech stack.
-

11. Features and Benefits of the Application

- **Unified Dashboard:** Combines everything into one view, reducing app-switching fatigue.
 - **Personalized Fitness Plan:** Does not provide generic advice; calculations are explicitly tailored to the user's specific biometric data.
 - **Diet & Macro Tracking:** Educates the user on nutritional splits, not just raw calories.
 - **Gamified Attendance System:** The psychological mechanism of "not wanting to break the streak" proves highly effective in building long-term fitness habits. Badges reward sustained consistency.
-

12. Why This Project is Effective

It solves the real-world problem of failure through complexity. By automating the math behind dieting and offering structured, easy-to-follow workout plans, it removes the cognitive load from the user. Integrating a psychological streak system directly into where they view their workouts ensures adherence. It acts as an automated, unbiased personal trainer in the user's pocket.

13. Future Enhancements

1. **AI-Based Recommendations:** Utilizing R's machine learning capabilities to adjust calorie goals dynamically if the user's weight plateaus.
 2. **Food Image Recognition:** Using computer vision APIs to scan a plate of food and estimate calories automatically.
 3. **Wearable Integration:** Direct syncing with Apple HealthKit or Google Fit APIs for real-time step and heart-rate data.
 4. **Social Leaderboards:** Adding a cloud backend (like Firebase) to allow users to compare streaks with friends.
-

14. Conclusion

FitTrack AI successfully integrates cross-platform mobile technology (Flutter) with advanced statistical backend processing (R/Plumber) to deliver a seamless, high-performance health tracking tool. By focusing on personalization, user experience, and the psychological aspects of habit building through gamification, this application provides a robust, scalable solution to modern personal fitness management.

(Page Break)

15. Viva Questions and Answers

Q1. Why did you choose Flutter for the frontend? Ans: Flutter allows us to build natively compiled applications for both mobile, web, and desktop from a single Dart codebase. It provides 60fps performance and highly customizable UI components, which was vital for our complex, animated dashboards.

Q2. Why did you use R programming for the backend instead of Python or Node.js? Ans: R is specifically designed for statistical and mathematical computing. Since the core logic of our app relies on processing biometric data through formulas (like Mifflin-St Jeor), R handled this perfectly. R also sets up the architecture perfectly for future machine learning scaling.

Q3. How did you connect R to the Flutter frontend? Ans: I used an R package called `plumber`. Plumber reads special annotations in the R script and turns those R functions into a RESTful API. Flutter then communicates with this API via HTTP POST requests using JSON.

Q4. What is BMR and how did you calculate it? Ans: BMR stands for Basal Metabolic Rate—the number of calories your body burns at rest. I calculated it using the Mifflin-St Jeor equation in R, which takes weight, height, age, and gender into account.

Q5. How does your attendance system encourage users? Ans: It uses gamification principles. By tracking consecutive days and displaying a "Streak Count" alongside visual achievements (badges), it leverages the psychological desire to maintain consistency to prevent the streak from resetting.

Q6. What happens to the streak if a user misses a day? Ans: The logic checks consecutive dates. If the loop detects a missing day between the last marked day and yesterday, the streak effectively breaks and recalculates from zero.

Q7. What database did you use and why? Ans: SQLite was used. It is a lightweight, embedded relational database that requires no separate server process. It allows the app to be fast and offline-first since health data needs to be accessible instantly.

Q8. What are "Macros" in your Diet module? Ans: Macros, or macronutrients, represent the three main categories of nutrients that provide caloric energy: Proteins, Carbohydrates, and Fats. The app calculates specific gram targets for each based on the user's total daily calories.

Q9. How do you calculate how much Protein a user needs? Ans: The system uses a general fitness guideline in the R script: roughly 2 to 2.2 grams per kilogram of body weight, ensuring sufficient protein for muscle maintenance and growth.

Q10. What is a REST API? Ans: Representational State Transfer (REST) is a software architectural style. It allows different systems (our Flutter App and R Server) to communicate over the internet using standard HTTP methods like GET and POST.

Q11. Explain state management in your Flutter app. Ans: State management is how the app handles changes to data (like adding a meal) and updates the UI. Tools like `Provider` or `setState()` manage this, ensuring that when the database is updated, widgets like the calorie ring instantly redraw.

Q12. What are the limitations of your current project? Ans: Currently, the R backend might require hosting on a server running an R environment (like Docker). Also, food logging requires manual calorie input rather than a searchable food database API.

Q13. How does the app handle offline capabilities? Ans: Because we use local storage/SQLite, if the external API is unreachable, the app still functions using the previously cached BMR/Calorie targets until

the connection is restored.

Q14. How would you prevent SQL Injection in your app? Ans: By using parameterized queries natively provided by the SQLite plugins, rather than manually concatenating string inputs into the SQL commands.

Q15. Why is there a difference in formulas for Male and Female BMR? Ans: Males generally have higher muscle mass and lower body fat percentages compared to females of the exact same height and weight, meaning their bodies burn more energy at rest natively. The constant in the formula adjusts for this physiological average.

Q16. What is TDEE? Ans: Total Daily Energy Expenditure. It is calculated by taking the BMR and multiplying it by an activity factor (e.g., 1.55 for moderate exercise) to estimate the total calories burned in a day including all movement.

Q17. How did you design the user interface? Ans: I utilized a dark-mode first design with high-contrast neon accents alongside glassmorphism (translucent blur effects) to make the data pop and give the app a premium, modern feel.

Q18. What was the most challenging part of this project? Ans: Structuring the communication safely between the Dart frontend and the R backend, ensuring data types matched (string to float conversions) so the mathematical models did not fail.

Q19. How did you structure the project files? Ans: It is structured into clearly separated distinct modules: UI components, State/Logic models, Database helper classes, and external API service handlers, adhering to clean architecture.

Q20. How will this project stand out from apps like MyFitnessPal? Ans: FitTrack AI is not just a food logger or just a workout app; it enforces discipline by linking nutritional tracking directly with an attendance streak system, putting accountability at the forefront.

[End of Document]